

AnimalCare — System Presentation Manuscript

Course context: IT22 Information Management — PIT (adjust as needed)

1. Introduction

System title

AnimalCare — Community Lost, Found, Rescue, and Adoption Web Application

Members and roles

Name	Role
[Member 1]	[e.g., Project lead / database & transactions]
[Member 2]	[e.g., Frontend / Angular]
[Member 3]	[e.g., Documentation / QA / admin features]

Replace bracketed entries with your actual team roster.

Overview of the system

AnimalCare is a web application for posting and browsing **lost**, **found**, **rescue**, and **adoption** listings. Users can register, sign in, create and manage posts, exchange **direct messages**, and—on adoption posts—**request adoption** so that one adopter can reserve a pet while the post owner can **approve or reject** the request. An **administrator** can oversee users, posts, comments, reservations, and a **transaction audit log** for accountability. The system is implemented as a single-page application with **role-based access** and **server-enforced** database rules (Firestore security rules).

2. Problem Statement

What problem does the system solve?

Pet-related information is often scattered across social media groups, chat applications, and informal channels. That fragmentation makes listings harder to **discover**, **verify**, and **coordinate**—especially for adoption, where only one family should hold an active reservation at a time. Without a dedicated system, teams face **duplicate reservations**, **lost context**, and **limited traceability** of important state changes.

Target users

- **General users** reporting lost or found animals, or promoting rescue and adoption listings.
 - **Adopters** seeking a clear, fair reservation process tied to a specific listing.
 - **Administrators** responsible for moderation, oversight, and operational visibility.
-

3. Objectives

General objective

To design and implement a web-based information system that centralizes animal-care listings and supports a **consistent adoption reservation workflow** with **auditable** multi-step database operations.

Specific objectives

1. Provide **authentication** (register, login, logout) with **unique email** enforcement.
2. Support **create, read, update, and delete** patterns for posts by type (lost, found, rescue, adoption) and for **comments**.
3. Implement **private messaging** between users, including read-state updates.

4. Implement the **adoption reservation** lifecycle: request, cancel, approve, reject—with **concurrency-safe** updates to posts and reservations.
 5. Deliver an **admin dashboard** with statistics and management views for posts, comments, users, reservations, and **transaction logs**.
 6. Document **database design** (entities, relationships, constraints) and **transactional behavior** (e.g., atomicity, consistency, isolation, durability) as required by the course rubric.
-

4. System Features

Core functionalities

- User **registration** and **login**; **role-based** navigation and route protection.
- **Feed** and **create-post** experiences across listing types.
- **Comments** on posts.
- **Direct messaging** between users.
- **Adoption reservations**: request to adopt, cancel, and owner **approve/reject**, with corresponding post reservation fields.
- **Administration**: dashboard metrics; moderation and listing management; **immutable-style** transaction audit trail in the UI.

Key modules (code organization)

- **Authentication and guards** — e.g., `AuthService`, auth-related route guards.
 - **Posts and feed** — features for lost/found, rescue, adoption, feed, and create-post.
 - **Messaging** — messages feature and `MessageService`.
 - **Transactions and reservations** — centralized transactional logic (e.g., `TransactionService`) using Firestore `runTransaction` and `writeBatch` where appropriate.
 - **Admin** — consolidated admin UI (statistics, tabs for entities and transaction log).
 - **Shared components** — e.g., post cards that surface reservation actions.
-

5. System Design

UML and diagrams

- **Use case diagram (recommended)**: Actors such as Guest, Registered User, Post Owner, Adopter, and Admin. Use cases include Register, Login, Create/Edit/Delete Post, Comment, Send Message, Request/Cancel Adoption, Approve/Reject Reservation, Browse Feed, Admin moderation, and View transaction audit.
- **ERD**: Relational-style model over Firestore collections, including **users**, **user_emails** (unique email sentinel), **posts**, **comments**, **reservations**, **messages**, and **transactions** (audit), with primary and foreign keys as documented in the project (`ACID.md`, §2).
- **Optional sequence diagram**: “Request adoption” showing read → validate → transactional writes to reservation and post → audit log.

Architecture

- **Client**: Single-page application built with **Angular** (TypeScript).
 - **Platform**: **Firebase Authentication** and **Cloud Firestore**.
 - **Security model**: **Firestore security rules** enforcing required fields, enumerations, immutability of identity fields where applicable, and append-only rules for audit documents.
 - **Integrity patterns**: Firestore **transactions** for multi-document updates that must succeed or fail together (e.g., reservation creation, registration with email uniqueness), and **atomic batches** for operations such as cascading deletes and related audit rows.
-

6. Live Demonstration (suggested script)

1. **Login** as a regular user; demonstrate **navigation** (feed, create post, messages).
2. **Create** a listing (e.g., adoption) and show it in the **feed** or detail view.
3. Using a **second account** or browser profile, **request adoption** and show reservation-related UI.

4. As the **post owner**, **approve** or **reject** the reservation and show updated status.
5. As **admin**, open the **dashboard**, review **reservations**, and open the **Transactions** tab to show **COMMITTED** versus **ROLLED_BACK** outcomes (and retries if two users race).
6. **Optional advanced demo:** Attempt simultaneous reservations on the same adoption post; show that only one commits and the other fails with a clear message and audit entry.
7. **Delete** a post that has comments and/or reservations; show that related data is removed in a coordinated way (atomic batch).

Prepare dedicated **demo accounts** before the presentation. Do not embed real passwords in handouts or slides.

7. Tools & Technologies Used

Layer	Technology
Language	TypeScript
Frontend	Angular (~21), Angular Router, Angular SSR-related packages as configured
Reactive streams	RxJS
Backend services	Firebase (Authentication, Firestore)
Configuration	Firebase project, <code>firestore.rules</code> , <code>firestore.indexes.json</code>
Tooling	Angular CLI, npm, Prettier, Vitest (as in project configuration)

Exact versions should match `animal-care-angular/package.json` at submission time.

8. Challenges Encountered

Customize this section with your team's real experience. Examples that often apply to this stack:

- **Concurrent adoption requests:** Required **Firestore transactions** and **optimistic concurrency** (e.g., monotonic `version` on posts) instead of naive read-then-write patterns.
 - **Email uniqueness on a document database:** Addressed with a **sentinel collection** (`user_emails`) and a **transactional** claim of the email document id.
 - **Cascading consistency:** Deleting a post together with comments, reservations, and audit metadata using `writeBatch` so the database does not end in a partial state.
 - **Security rules:** Iterating on **Firestore rules** for enums, required fields, and **immutable audit** documents until behavior matched requirements.
 - **Team workflow:** Coordinating schema, service boundaries, and reviews via **git** and shared documentation.
-

9. Conclusion & Recommendations

Conclusion

AnimalCare delivers a **structured, role-aware** platform for pet listings and adoption coordination. The combination of **client services**, **Firestore transactions and batches**, and **security rules**, together with an **audit-oriented transaction log**, supports **data integrity** and **traceability** appropriate for an information-management systems project.

Recommendations

- Add **notifications** (email or push) for reservation status changes.
 - Integrate **Firebase Storage** for photo attachments on listings.
 - Provide **admin export** (CSV/PDF) for reports.
 - Expand **automated end-to-end tests** for critical user journeys.
 - Strengthen **privacy** messaging and policies for contact information displayed on listings.
-

Appendix — Suggested PowerPoint slide list

1. Title — AnimalCare; course; group; date
2. Agenda — align with instructor sections
3. Team — names and roles
4. System overview — one paragraph + screenshot
5. Problem statement
6. Target users
7. General and specific objectives
8. Features — bullet overview
9. Key modules — diagram or folder map
10. Use case diagram
11. ERD
12. Architecture — Angular + Firebase + rules
13. Integrity — transactions, batches, audit (brief)
14. Live demo — numbered steps
15. Tech stack table
16. Challenges and solutions
17. Conclusion
18. Recommendations — Q&A

Generated for the AnimalCare project repository. ERD and ACID details: see `ACID.md` in the project root.